

REDBACKINTERPMETHOD

January 16, 2026

```
[1]: import redback
print(redback.__version__)

import numpy as np
import matplotlib.pyplot as plt
from redback.model_library import all_models_dict

from astropy.cosmology import Planck18 as cosmo

import redback.interaction_processes as ip
import redback.sed as sed
import redback.photosphere as photosphere

import astropy.units as uu

import extinction
from extinction import ccm89, fitzpatrick99, apply, remove
from scipy.interpolate import RegularGridInterpolator
import sncosmo

import inspect
from redback.transient_models import supernova_models

#import lambda to nu
from redback.transient_models.supernova_models import lambda_to_nu
```

No module named 'lalsimulation'

lalsimulation is not installed. Some EOS based models will not work. Please use bilby eos or pass your own EOS generation class to the model

14:12 bilby INFO : Running bilby version: 2.3.0

14:12 redback INFO : Running redback version: 1.12.1

1.12.1

```
[2]: #define model fro redback

def sn1998bw_template(time, redshift, amplitude, **kwargs):
    """
```

A wrapper to the SN1998bw template. Only valid between 1100-11000 Angstrom and 0.01 to 90 days post explosion in rest frame

Parameters

time

time in days in observer frame (post explosion)

redshift

redshift

amplitude

amplitude scaling factor, where 1.0 is the original brightness of SN1998bw; and f_lambda is scaled by this factor

kwargs

Additional keyword arguments required by redback.

frequency

Required if output_format is 'flux_density'. frequency to calculate ν (Must be same length as time array or a single number).

bands

Required if output_format is 'magnitude' or 'flux'.

output_format

'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'

cosmology

Cosmology to use for luminosity distance calculation. Defaults to Planck18. Must be a astropy.cosmology object.

Returns

set by output format - 'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'

"""

import sncosmo

model = sncosmo.Model(source='v19-1998bw')

original_redshift = 0.0085

cosmology = kwargs.get("cosmology", cosmo)

original_dl = (43*uu.Mpc).to(uu.cm).value

From roughly matching to Galama+ or Clocchiatti+1998bw light curves

original_peak_time = 15

model.set(z=original_redshift, t0=original_peak_time)

model.set_source_peakmag(14.25, band='bessellb', magsys='ab')

tts = np.geomspace(0.01, 90, 200)

lls = np.linspace(1620, 11000, 300)

f_lambda = model.flux(tts, lls) #erg/s/cm²/Angstrom.

l_lambda = f_lambda * 4 * np.pi * original_dl**2 # erg/s/Angstrom

We consider this the rest frame spectrum of 1998bw. Now we can redshift it and scale it.

```

time_obs = tts * (1 + redshift)
lambda_obs = lls * (1 + redshift)
dl_new = cosmology.luminosity_distance(redshift).cgs.value
f_lambda_obs = l_lambda / (4 * np.pi * dl_new**2)
f_lambda_obs = amplitude * f_lambda_obs * (1 + redshift) # accounting for
↳bandwidth stretching
f_lambda_obs = f_lambda_obs * uu.erg / uu.s / uu.cm ** 2 / uu.Angstrom
if kwargs['output_format'] == 'flux_density':
    frequency = kwargs['frequency']
    # work in obs frame
    ff_array = lambda_to_nu(lambda_obs)

    # Convert flux density to mJy
    fmjy = f_lambda_obs.to(uu.mJy, equivalencies=uu.
↳spectral_density(wav=lambda_obs * uu.Angstrom)).value
    # Create interpolator on obs frame grid
    flux_interpolator = RegularGridInterpolator(
        (time_obs, ff_array),
        fmjy,
        bounds_error=False,
        fill_value=0.0)

    # Prepare points for interpolation
    if isinstance(frequency, (int, float)):
        frequency = np.ones_like(time) * frequency

    # Create points for evaluation
    points = np.column_stack((time, frequency))

    # Return interpolated flux density with (1+z) correction for observer
↳frame
    return flux_interpolator(points)
    elif kwargs['output_format'] == 'spectra':
        return namedtuple('output', ['time', 'lambdas',
↳'spectra'])(time=time_obs,

        lambdas=lambda_obs,

        spectra=f_lambda_obs)
    else:
        return sed.get_correct_output_format_from_spectra(time=time,
↳time_eval=time_obs,

        spectra=f_lambda_obs,

        lambda_array=lambda_obs,

        **kwargs)

```

```
[3]: #specific GRB event info (to compare to 1998bw)

#GRB 231117A
# event_name = '231117A'
# red = 0.257
# time = np.array(1.32) #lowest possible time - short GRB - increase to 0.1
# freq = 4.85800e+14 #match freq to redback filter band
# AB_mag = 20.86
# #specify the filter (conversion if needed)
# #dustmaps does the correct magnitude correction !!
# a_v = 0.186 #E(B-V) x R_v

# wavelength= np.array([6175.58000])

[ ]: event_name = '171205A'
AB_mag = 17.7
red = 0.0368
time = np.array(10.98)
freq = 3.74100e+14
# band = 'besselli'
a_v = 0.05 * 3.1 #E(B-V) x R_v

#extinction along the line-of-sight of E(B-V) = 0.05 mag nor the one intrinsic
↳to the host with a value of E(B-V)int = 0.02
#get wavelength from filter tables redback
#ensure i am always matching my wavelength to my frequency band if i am
↳changing it !!
wavelength = np.array([8020.14000]) #angstroms

[4]: #interpolate sn1998bw model to find flux at precise time, filter, redshift
sn_1998bw = sn1998bw_template(time=time,
redshift=red,
amplitude=1.0,
frequency=freq,
output_format='flux_density')

print("Flux density of 1998bw =", sn_1998bw[0], "mJy")

Flux density of 1998bw = 0.003761838346985619 mJy

[5]: #convert AB magnitude of arbitrary GRB event --> flux density (mJy)
def calc_flux_density_from_ABmag(AB_mag):
    """
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
```

```

        :return: flux density
        """
        return (AB_mag * uu.ABmag).to(uu.mJy)

flux_density_event = calc_flux_density_from_ABmag(AB_mag)

print("This is the flux density of GRB event", event_name, f"without_
↳extinction correction: {flux_density_event:.5f}")

dereddened_flux_event = extinction.remove(fitzpatrick99(wavelength, a_v, 3.
↳1), flux_density_event)

#dereddened flux is now a 1-d array so display the first element
print(f"This is the extinction corrected GRB event flux density found using_
↳fitzpatrick99: {dereddened_flux_event[0]:.5f}")

```

This is the flux density of GRB event 231117A without extinction correction:
0.01644 mJy
This is the extinction corrected GRB event flux density found using
fitzpatrick99: 0.01896 mJy

```

[6]: '''#if host = prominent:
mag_host = 0
#band = I

#conversion:
AB_mag_host = mag_host #+ 0.45

#this doesnt change the ratio though !!

#if host is not prominent, let AB_mag_host = 0 and run cell as normal:

def calc_flux_density_from_ABmag(AB_mag_host):
    """
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
    :return: flux density
    """
    return (AB_mag_host * uu.ABmag).to(uu.mJy)

flux_density_host = calc_flux_density_from_ABmag(AB_mag_host)

print("This is the flux density of GRB host", f": {flux_density_host:.5f}")

#now subtract this from the de-reddened to get the host-corrected flux density

```

```

host_corrected_flux_density = dereddened_flux_event - flux_density_host

print("This is the flux density of the GRB event ", event_name, "corrected for_
↳ its bright host galaxy:", host_corrected_flux_density)'''

```

```

[6]: '#if host = prominent: \nmag_host = 0\n#band = I \n\n#conversion: \nAB_mag_host
= mag_host #+ 0.45\n\n#this doesnt change the ratio though !! \n\n#if host is
not prominent, let AB_mag_host = 0 and run cell as normal:\n\nndef
calc_flux_density_from_ABmag(AB_mag_host):\n    """\n    Calculate flux density
from AB magnitude assuming monochromatic AB filter\n\n    :param magnitudes: AB
magnitude values\n    :return: flux density\n    """\n    return (AB_mag_host *
uu.ABmag).to(uu.mJy)\n\nflux_density_host =
calc_flux_density_from_ABmag(AB_mag_host)\n\nprint("This is the flux density of
GRB host", f": {flux_density_host:.5f}")\n\n#now subtract this from the de-
reddened to get the host-corrected flux density\n\nhost_corrected_flux_density =
dereddened_flux_event - flux_density_host\n\nprint("This is the flux density of
the GRB event ", event_name, "corrected for its bright host galaxy:",
host_corrected_flux_density)'

```

```

[8]: #final calculation - taking ratio of both fluxes
ratio = dereddened_flux_event[0] /sn_1998bw[0]
print(f"Corrected Final Flux Ratio: {ratio.value:.3f}")

#SN1998 model is underpredicting for some reason ?

```

The final flux density ratio result of $F_{\text{GRB}} / F_{1998\text{bw}} = 5.041$ mJy
Corrected Final Flux Ratio: 5.041

```

[25]: #STEP 1: define event

event_name = '171205A'
AB_mag = 17.7
redshift = 0.0368
epoch = np.array([10.98])
# band = 'besselli'
frequency = 4.00500e+14
a_v = 0.05 * 3.1 #E(B-V) x R_v

```

```

[26]: #STEP 2: Convert GRB event AB magnitude into flux density using redback
def calc_flux_density_from_ABmag(AB_mag):
    """
    Calculate flux density from AB magnitude assuming monochromatic AB filter

    :param magnitudes: AB magnitude values
    :return: flux density
    """
    return (AB_mag * uu.ABmag).to(uu.mJy)

```

```
flux_density_event = calc_flux_density_from_ABmag(AB_mag)

print("This is the flux density of GRB event", event_name ,f"without_
↳extinction correction: {flux_density_event:.3f}")
```

This is the flux density of GRB event 171205A without extinction correction:
0.302 mJy

```
[27]: #STEP 3: Use fitzpatrick99 extinction function (if applicable) to find_
↳de-reddened value for GRB flux density
#to get wavelength for GRB event, use redback tables
#wavelength must be an array
wavelength = np.array([7489.98000]) #angstroms -- r-band wavelength
dereddened_flux_event = extinction.remove(fitzpatrick99(wavelength, a_v, 3.
↳1),flux_density_event)
#dereddened flux is now a 1-d array so display the first element
print(f"This is the extinction corrected GRB event flux density found using_
↳fitzpatrick99: {dereddened_flux_event[0]:.3f}")
```

This is the extinction corrected GRB event flux density found using
fitzpatrick99: 0.330 mJy

```
[28]: #STEP 4: load in model from redback

#calculate 1-D array of flux densities of 1998bw over time
#bug fix : ensure time is the same or related to the observed time
# i was working in two different frames which caused a 0 error

#define lambda_to_nu outside of the function: does this make the previous_
↳frequencies_Hz obsolete ?

def lambda_to_nu(wavelength_angstrom):
    """ Converts wavelength in Angstroms to frequency in Hz """
    c = 299792458 # speed of light in m/s
    return c / (wavelength_angstrom * 1e-10)

#change the inner workings of the sn1998bw def
def sn1998bw_template(time, redshift, amplitude, **kwargs):
    """
    A wrapper to the SN1998bw template. Only valid between 1100-11000 Angstrom_
    ↳and 0.01 to 90 days post explosion in rest frame

    Parameters
    -----
    time
        time in days in observer frame (post explosion)
    redshift
```

```

    redshift
    amplitude
        amplitude scaling factor, where 1.0 is the original brightness of
        ↪ SN1998bw; and f_lambda is scaled by this factor
    kwargs
        Additional keyword arguments required by redback.
    frequency
        Required if output_format is 'flux_density'. frequency to calculate
        ↪ (Must be same length as time array or a single number).
    bands
        Required if output_format is 'magnitude' or 'flux'.
    output_format
        'flux_density', 'magnitude', 'spectra', 'flux', 'sncosmo_source'
    cosmology
        Cosmology to use for luminosity distance calculation. Defaults to
        ↪ Planck18. Must be a astropy.cosmology object.
    Returns
    -----
        set by output format - 'flux_density', 'magnitude', 'spectra', 'flux',
        ↪ 'sncosmo_source'

    """
    import sncosmo
    model = sncosmo.Model(source='v19-1998bw')
    original_redshift = 0.0085 #redshift of 1998bw
    cosmology = kwargs.get("cosmology", cosmo)
    original_dl = (43*uu.Mpc).to(uu.cm).value

    # From roughly matching to Galama+ or Clocchiatti+1998bw light curves
    original_peak_time = 15
    model.set(z=original_redshift, t0=original_peak_time)
    model.set_source_peakmag(14.25, band='bessellb', magsys='ab')
    lls = np.linspace(1620, 11000, 300)
    f_lambda = model.flux(time, lls) #erg/s/cm^2/Angstrom.
    l_lambda = f_lambda * 4 * np.pi * original_dl**2 # erg/s/Angstrom

    # We consider this the rest frame spectrum of 1998bw. Now we can redshift
    ↪ it and scale it.
    time_obs = time * (1 + redshift)
    lambda_obs = lls * (1 + redshift)
    dl_new = cosmology.luminosity_distance(redshift).cgs.value
    f_lambda_obs = l_lambda / (4 * np.pi * dl_new**2)
    f_lambda_obs = amplitude * f_lambda_obs * (1 + redshift) # accounting for
    ↪ bandwidth stretching
    f_lambda_obs = f_lambda_obs * uu.erg / uu.s / uu.cm ** 2 / uu.Angstrom
    if kwargs['output_format'] == 'flux_density':

```



```

frequency = kwargs['frequency']
# work in obs frame
ff_array = lambda_to_nu(lambda_obs)

# Convert flux density to mJy
fmjy = f_lambda_obs.to(uu.mJy, equivalencies=uu.
↪spectral_density(wav=lambda_obs * uu.Angstrom)).value
# Create interpolator on obs frame grid
flux_interpolator = RegularGridInterpolator(
    (time_obs, ff_array),
    fmjy,
    bounds_error=False,
    fill_value=0.0)

# Prepare points for interpolation
if isinstance(frequency, (int, float)):
    frequency = np.ones_like(time) * frequency

# Create points for evaluation
points = np.column_stack((time, frequency))

# Return interpolated flux density with (1+z) correction for observer
↪frame
return flux_interpolator(points)
elif kwargs['output_format'] == 'spectra':
    return namedtuple('output', ['time', 'lambdas', '
↪spectra'])(time=time_obs,

↪lambdas=lambda_obs,

↪spectra=f_lambda_obs)
else:
    return sed.get_correct_output_format_from_spectra(time=time,
↪time_eval=time_obs,

↪lambda_array=lambda_obs,

**kwargs)

```

```

[29]: result = sn1998bw_template(
    time=epoch,
    redshift=redshift,
    amplitude=1.0,
    output_format='flux_density',

```

```

    frequency=frequency, # TRY CENTRAL FREQ NOT -- singular frequency from
    ↪redback tables I-band effective width in Hz OR multiple values ? WHICH ONE
    ↪WORKS ? --
    cosmology=cosmo
)

f_1998bw_interpolated = result[0] #flux for 1998bw in mJy (float?)

# result[0] is now exactly the flux at day 10.5
print(f"Flux density at day {epoch}: {f_1998bw_interpolated} mJy")

#SKIPPED STEP 6 -- INTERPOLATED IN ONE GO

'''FINAL RATIO:'''
#STEP 7: take value from STEP 3 and STEP 6 in ratio with one another to get
    ↪final result of flux ratio

f_1998bw_ratio = dereddened_flux_event / f_1998bw_interpolated

print(f"The final flux density ratio result of F_GRB / F_1998bw =
    ↪{f_1998bw_ratio[0]:.3f}")

```

Flux density at day [10.98]: 0.0 mJy

The final flux density ratio result of F_GRB / F_1998bw = inf mJy